

FORMATION R & RSTUDIO



GeoAgriData Analytics (Solution)

JOUR 1

Introduction en RStudio

Module 1 : Fondements de R et RStudio

Organisateur :

GeoAgri Data Solution (GADASol)
Projet GeoAgri Data Analytics

17 Mai – 17 Juin 202-

Goma, Nord-Kivu, RDC

www.gadatrainng.org

<https://gada-training.github.io/gada-training/>

1. Présentation de R et RStudio

1.1 Pourquoi utiliser R pour l'analyse des données ?

R est un langage de programmation puissant et largement utilisé dans l'analyse statistique et la science des données. Il est particulièrement adapté aux domaines de l'agriculture, de l'environnement et de la santé publique dans le cadre d'une approche One Health.

- ✓ **Gratuit et open-source** : Contrairement à des logiciels comme SPSS ou SAS, R est accessible à tous sans coût.
- ✓ **Polyvalent** : Il permet l'analyse statistique, la modélisation, le machine learning, la manipulation de données et la visualisation.
- ✓ **Large communauté** : R dispose d'une vaste communauté qui développe constamment de nouveaux packages.
- ✓ **Personnalisable** : Contrairement à Excel, R permet d'automatiser et de personnaliser les traitements sur de grandes bases de données.
- ✓ **Reproductibilité** : Grâce aux scripts et à RMarkdown, il est possible de reproduire et documenter facilement les analyses.
- ✓ **Visualisation avancée** : Avec ggplot2, R permet de créer des graphiques professionnels et interactifs.

1.2 L'interface de RStudio

RStudio est un environnement de développement intégré (IDE) qui facilite l'utilisation de R. Son interface est divisée en quatre panneaux principaux :

Console	(bas gauche) — C'est ici que l'on tape les commandes et que R exécute le code. Raccourci : Ctrl + Enter pour lancer une ligne.
Script	(haut gauche) — Permet d'écrire et de sauvegarder du code R dans des fichiers .R. Lancer une ligne : Ctrl + Enter.
Environnement / Historique	(haut droit) — Environnement : affiche les objets stockés en mémoire (dataframes, variables, listes...) History : conserve l'historique des commandes exécutées.
Plots / Packages / Files / Help	(bas droit) — Plots : graphiques générés Packages : bibliothèques installées Files : naviguer dans le système de fichiers Help : documentation des fonctions

1.3 Installation et chargement des packages

Un package est un ensemble de fonctions et de données complémentaires à R.

□ 1. Installer un package

Exemple : installer ggplot2 pour la visualisation

```
install.packages("ggplot2")
```

□ 2. Charger un package

```
library(ggplot2)
```

□ 3. Voir la liste des packages installés

```
installed.packages()
```

□ 4. Vérifier si un package est installé

```
if (!require("ggplot2")) install.packages("ggplot2")  
library(ggplot2)
```

□ 5. Mettre à jour tous les packages

```
update.packages()
```

□ Exercice pratique — Découverte de RStudio

Tâche 1 : Ouvrir RStudio et repérer les 4 panneaux principaux.

Tâche 2 : Installer et charger les packages ggplot2 et dplyr.

Tâche 3 : Afficher la liste des packages installés.

Tâche 4 : Nettoyer l'environnement et la console avec `rm(list = ls())` et `cat("\014")`.

2. Les bases du langage R

2.1 Les types de données en R

En R, il existe plusieurs types de données fondamentales que l'on peut manipuler :

Type de données	Description	Exemples
Numérique (numeric, integer)	Valeurs numériques entières ou décimales	10, 3.14, -25
Caractère (character)	Texte, représenté entre guillemets	"Tomate", "Banane"
Facteur (factor)	Variables catégorielles (modalités)	"Petit", "Moyen", "Grand"
Logique (logical)	Valeurs booléennes (Vrai/Faux)	TRUE, FALSE

□ Vérifier le type d'une variable

```
x <- 10
class(x)    # Retourne "numeric"
y <- "Tomate"
class(y)    # Retourne "character"
z <- TRUE
class(z)    # Retourne "logical"
```

□ Conversion entre types de données

```
as.numeric("5")    # Convertir en numérique
as.character(10)    # Convertir en caractère
as.logical(0)       # 0 -> FALSE, tout autre nombre -> TRUE
```

2.2 Création et manipulation des objets

En R, les données sont stockées sous forme d'objets. Les principaux objets sont : vecteurs, matrices, dataframes et listes.

1 □ Vecteurs — structure de base en R

Un vecteur est une collection de valeurs du même type.

```
# Créer un vecteur
vecteur <- c(1, 2, 3, 4, 5)           # Vecteur numérique
noms    <- c("Banane", "Tomate", "Maïs") # Vecteur de caractères

# Manipuler un vecteur
length(vecteur)  # Nombre d'éléments
vecteur[2]       # Accéder au 2ème élément
vecteur[1:3]     # Extraire les trois premiers éléments
```

2▣ Matrices — tableaux à deux dimensions

Une matrice est une structure en deux dimensions (lignes et colonnes) où toutes les valeurs doivent être du même type.

```
# Créer une matrice 3x3
matrice <- matrix(1:9, nrow = 3, ncol = 3)
matrice

# Accéder aux éléments
matrice[2, 3] # 2ème ligne, 3ème colonne
matrice[, 1]  # 1ère colonne entière
matrice[1, ]  # 1ère ligne entière
```

3▣ Dataframes — tableaux de données

Un dataframe est une structure tabulaire similaire à une feuille Excel, avec des colonnes pouvant contenir différents types de données.

```
# Créer un dataframe
df <- data.frame(
  Culture      = c("Banane", "Tomate", "Maïs"),
  Rendement    = c(12.5, 8.3, 15.2),
  Irrigation   = c(TRUE, FALSE, TRUE)
)

# Manipuler un dataframe
dim(df)       # Nombre de lignes et colonnes
str(df)       # Structure du dataframe
df$Culture    # Accéder à une colonne
df[1, ]       # Première ligne
df[, 2]       # Deuxième colonne
df[1, 2]      # Premier élément de la 2ème colonne
```

4▣ Listes — objets complexes

Une liste peut contenir différents types d'objets : vecteurs, matrices, dataframes, etc.

```
# Créer une liste
ma_liste <- list(
  nom      = "Expérience agricole",
  donnees  = df,
  notes    = c(10, 20, 15)
)

# Accéder aux éléments
ma_liste$nom      # Élément "nom"
ma_liste$donnees  # Le dataframe
ma_liste$notes[2] # 2ème élément du vecteur "notes"
```


3. Opérations de base en R

3.1 Opérations arithmétiques

```
a <- 10 ; b <- 3
a + b      # Addition      -> 13
a - b      # Soustraction  -> 7
a * b      # Multiplication -> 30
a / b      # Division      -> 3.333...
a ^ b      # Puissance     -> 1000
a %% b     # Modulo (reste) -> 1
```

3.2 Opérations logiques

```
x <- 5 ; y <- 10
x > y      # FALSE
x < y      # TRUE
x == y     # FALSE
x != y     # TRUE
(x < y) & (x > 2) # TRUE (ET logique)
(x < y) | (x > 10) # TRUE (OU logique)
```

3.3 Conditions (if, else, ifelse)

```
age <- 20
if (age >= 18) {
  print("Majeur")
} else {
  print("Mineur")
}

# ifelse() pour un traitement rapide sur un vecteur
note <- c(15, 8, 18, 5, 12)
resultat <- ifelse(note >= 10, "Réussi", "Échoué")
resultat
```

4. Utilisation et création de fonctions

4.1 Fonctions de base

```
mean(c(1, 2, 3, 4, 5)) # Moyenne -> 3
sum(c(1, 2, 3, 4, 5))  # Somme   -> 15
```

```
sd(c(1, 2, 3, 4, 5))      # Écart-type -> 1.58
```

4.2 Création de fonctions personnalisées

On peut créer ses propres fonctions en R pour automatiser des calculs répétitifs.

Exemple 1 : Convertir des kg en grammes

```
convertir_kg_en_g <- function(kg) {  
  g <- kg * 1000  
  return(g)  
}  
convertir_kg_en_g(2.5)    # Résultat : 2500
```

Exemple 2 : Calculer l'aire d'un rectangle

```
aire_rectangle <- function(longueur, largeur) {  
  return(longueur * largeur)  
}  
aire_rectangle(5, 3)      # Résultat : 15
```

□ Exercice pratique — Objets et fonctions

Créer un vecteur avec les rendements de 5 cultures et calculer la moyenne.

Créer un dataframe avec les cultures, les rendements et l'irrigation.

Créer une fonction qui convertit une température de Celsius en Fahrenheit.

Formule : $F = (C \times 9/5) + 32$

✓Récapitulatif du Jour 1

- ✓ Comprendre les types de données en R (numeric, character, factor, logical)
- ✓ Créer et manipuler des vecteurs, matrices, dataframes et listes
- ✓ Effectuer des opérations arithmétiques et logiques
- ✓ Utiliser et créer des fonctions personnalisées
- ✓ Maîtriser l'environnement RStudio (panneaux, packages, console)

□ **Prochaine étape — Jour 2 : Importation et manipulation des données** □

□ **Contacts GADA Training**

- Goma, Nord-Kivu, RDC
- +243 898 383 148
- ✉ Gadasol25@gmail.com
- www.gadatrainning.org

□ **Calendrier de formation**

Semaine 1 : Fondements statistiques et SI
Semaine 2 : Modélisation spatiale et géographie santé
Semaine 3 : Intégration One Health avancée
Semaine 4 : Projet intégré et ingénierie de décision